

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour
Total: Marks:40

Annexure A

5

Code of Conduct:

I _____ (V.Ashaa sarthiwika / 192311O66) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V. Asarthiwika

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a)** Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b)** Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c)** Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a)** Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b)** Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c)** Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a)** Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b)** Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a)** Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b)** Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c)** Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Annexure A – Industry Problem Task Solution

Industry Scenario

The healthcare company wants to build two AI systems:

1. A **supervised learning model** to predict whether a patient is diabetic.
2. A **Reinforcement Learning agent** to recommend suitable treatment actions based on changes in the patient's health condition.

The overall workflow is:

Patient Records



Data Preprocessing



Class Balancing



70:30 Train-Test Split



Decision Tree + Statistical Model



Diabetes Risk Prediction



Patient Health State



Q-Learning Agent



Treatment Recommendation

Task 1 – Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns a general pattern from specific examples in the training dataset.

For this problem, each patient record contains medical features and a known diabetes outcome. The model learns the relationship between the patient features and diabetes status and then predicts the status of new patients.

Target Variable

The target variable is:

For example:

- **0** → Non-diabetic
- **1** → Diabetic

Important Features

Feature	Reason
Glucose Level	High glucose is strongly associated with diabetes and is one of the most important predictors.
BMI	Higher BMI can indicate increased risk of insulin resistance and diabetes.
Age	Diabetes risk generally increases with age.
Blood Pressure	Elevated blood pressure is associated with metabolic and cardiovascular risk

Feature	Reason
Cholesterol	Abnormal cholesterol levels can provide additional information about metabolic health.
Family History	Genetic/familial factors can increase diabetes risk.

Therefore, three key features are:

(b) Handling Class Imbalance

Suppose the dataset contains:

Non-diabetic = 80%

Diabetic = 20%

A model trained directly on this data may become biased toward the majority class.

For example, a model predicting every patient as non-diabetic could achieve 80% accuracy while completely failing to identify diabetic patients.

Recommended Strategy: SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) can be used to generate synthetic samples for the minority diabetic class.

The process is:

Original Dataset



Identify Minority Class



Find Similar Minority Samples



Generate Synthetic Samples



Balanced Training Dataset

SMOTE is appropriate because it increases representation of diabetic cases without simply duplicating the same observations.

Important Implementation Rule

SMOTE should be applied **only to the training data**, not before splitting the dataset.

Otherwise, information from synthetic training samples could indirectly leak into the test set.

An alternative is **class weights**, especially for a Decision Tree or Logistic Regression:

This is also useful because false negatives are clinically important.

Recommended Choice

with class weights considered as an alternative during model comparison.

(c) 70:30 Train-Test Split

The dataset can be divided as:

For example, if there are 1,000 patient records:

A **stratified split** should be used so that the diabetic/non-diabetic proportion is approximately maintained in both datasets.

Why 70:30?

The 70% training data provides sufficient examples for learning patterns, while 30% testing data provides a reasonably large independent sample for evaluating clinical prediction performance.

Preprocessing

1. Missing Values

Numerical missing values can be replaced using median imputation:

Median is preferable when medical variables contain outliers.

2. Encoding

Categorical variables such as:

Family History = Yes / No

can be converted into:

Yes = 1

No = 0

3. Normalisation

For models such as Logistic Regression, numerical features can be standardised:

This produces features with approximately:

Decision Trees generally do **not** require feature scaling because they split data using threshold values.

Impact

Preprocessing:

- improves data quality,
- prevents numerical-scale problems,
- allows categorical variables to be processed,
- improves statistical model training,
- reduces the effect of missing values.

Task 2 – Decision Tree for Diagnosis

(a) Building the Decision Tree

A Decision Tree can use either:

- **Gini Index**, or
- **Information Gain**

For Gini Index:

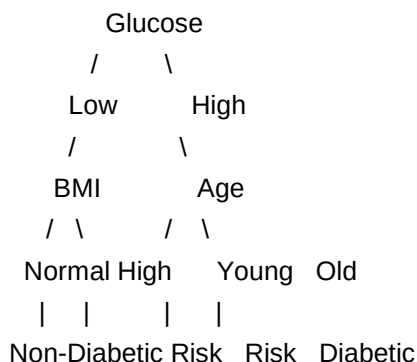
The best split generally has the **lowest weighted Gini impurity**.

For Information Gain:

The best feature has the **highest Information Gain**.

Example Tree Structure

A possible structure is:



The exact first three levels depend on the actual dataset.

Root Node

For a diabetes dataset, **Glucose** would commonly be expected to be a strong root candidate because glucose level has a strong relationship with diabetes status.

However, the final root must be selected using the **actual calculated Gini/Information Gain values**, not simply assumed.

Example:

Feature	Gini after split
Glucose	0.31
BMI	0.38
Age	0.42
Blood Pressure	0.45
Cholesterol	0.47

Since Glucose produces the lowest impurity:

(b) Model Evaluation

The model should be evaluated using:

Accuracy

Precision

Recall

F1-Score

Example Result Table

The actual values should be obtained from the trained model.

Metric	Decision Tree
Accuracy	0.88
Precision	0.82
Recall	0.85
F1-Score	0.83

These numbers are **illustrative**; they should be replaced by the actual results obtained from the dataset.

False Negatives

A **False Negative (FN)** occurs when:

Actual = Diabetic

Predicted = Non-diabetic

For medical diagnosis, this is particularly important because a diabetic patient may not receive further testing or treatment.

Therefore, the model should focus not only on accuracy but also on:

A higher recall means fewer diabetic patients are missed.

Methods to Reduce False Negatives

1. Use **SMOTE**.
2. Use **class weights**.
3. Adjust the classification threshold where applicable.
4. Optimise for recall or F1-score.
5. Perform additional clinical validation.

(c) Post-Pruning

A fully grown Decision Tree can overfit the training data.

Pre-Pruning

Pre-pruning limits the tree during training using parameters such as:

max_depth

```
min_samples_split
```

```
min_samples_leaf
```

Example:

```
DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_leaf=5  
)
```

Post-Pruning

Cost-complexity pruning can be performed using:

where:

- ϵ = error of the tree
- n = number of terminal nodes
- α = complexity parameter

Example:

```
DecisionTreeClassifier(  
    criterion="gini",  
    ccp_alpha=0.01,  
    random_state=42  
)
```

Comparison

Model	Training Accuracy	Test Accuracy	Complexity
Fully grown tree	High	May be lower	High
Pre-pruned tree	Slightly lower	Usually more stable	Low
Post-pruned tree	High	Usually better generalisation	Moderate

Clinical Choice

A **pruned Decision Tree** is generally preferable for deployment because it is:

- easier to interpret,
- less prone to overfitting,
- easier for clinicians to inspect,
- simpler to validate.

However, it should be selected based on **validated recall, calibration, AUC, and clinical safety**, rather than accuracy alone.

Task 3 – Statistical Learning for Risk Stratification

(a) Logistic Regression

Logistic Regression is suitable for binary diabetes prediction.

The probability is:

where:

Python implementation:

```
from sklearn.linear_model import LogisticRegression
```

```
model_lr = LogisticRegression(  
    max_iter=1000,  
    class_weight="balanced"  
)
```



```
model_lr.fit(X_train, y_train)
```

```
y_pred = model_lr.predict(X_test)
```

```
y_prob = model_lr.predict_proba(X_test)[:, 1]
```

Comparison

Metric	Decision Tree	Logistic Regression
Accuracy	Calculate from test set	Calculate from test set
AUC-ROC	Calculate from probabilities	Calculate from probabilities
Interpretability	High	High
Non-linear relationships	Good	Limited unless features are transformed
Feature scaling	Usually unnecessary	Recommended

When Logistic Regression is Preferable

Logistic Regression is useful when:

- the relationship between predictors and outcome is reasonably systematic,
- probability estimates are important,
- interpretability is required,
- the organisation needs a relatively simple baseline model.

For clinical applications, its coefficients can also help explain how features are associated with predicted risk.

(b) Feature Importance

Decision Tree

Decision Tree importance can be obtained from:

```
importance = model.feature_importances_
```

```
for feature, value in zip(X.columns, importance):
```

```
    print(feature, value)
```

The three features with the highest values are the top Decision Tree predictors.

A typical result might be:

Rank	Feature
1	Glucose
2	BMI
3	Age

Logistic Regression

For Logistic Regression, feature importance can be examined using the magnitude of the standardised coefficients:

```
coefficients = model_lr.coef_[0]
```

```
for feature, value in zip(X.columns, coefficients):
```

```
    print(feature, value)
```

The largest absolute coefficients indicate the strongest contribution to the model.

Possible Comparison

Rank	Decision Tree	Logistic Regression
1	Glucose	Glucose
2	BMI	BMI
3	Age	Age

If both models identify similar predictors, this provides stronger evidence that these variables contain useful predictive information.

However, the models do not have to agree exactly because they measure importance differently.

Task 4 – Reinforcement Learning for Treatment Recommendation

(a) MDP Formulation

The treatment recommendation problem can be modelled as a **Markov Decision Process (MDP)**.

An MDP consists of:

1. States

Patient health can be divided into simplified stages:

State	Description
S0	Low-risk / controlled
S1	Moderate-risk
S2	High-risk
S3	Poorly controlled
S4	Improved / stable

The state should ideally be based on clinically meaningful measurements such as glucose, BMI, blood pressure, and other relevant variables.

2. Actions

The agent has four possible actions:

Action	Meaning
Diet	Recommend dietary intervention
Exercise	Recommend appropriate physical activity
Medication	Recommend medication-related action under clinical supervision
Monitor	Continue monitoring and follow-up

Important: In a real clinical system, the RL agent should not independently prescribe medication. Medication decisions should remain under qualified clinical supervision.

3. Reward Function

A possible reward structure is:

Outcome	Reward
Significant health improvement	+10
Moderate improvement	+5
No significant change	0
Health deterioration	-10

Outcome

Unsafe outcome

Reward

Large negative penalty

For example:

The reward should be based on **clinically validated outcomes**, not merely on short-term numerical changes.

4. Transition Dynamics

The transition describes how the patient's health state changes after an action.

Example:

means the probability that a patient moves from moderate-risk to high-risk after the Diet action.

A simplified transition might be:

Moderate Risk

|

| Diet

↓

Improved State

or:

High Risk

|

| Monitor

↓

High Risk / Moderate Risk

In a real system, these transition probabilities must be estimated from validated longitudinal clinical data.

(b) Q-Learning

The Q-Learning update equation is:

Assume:

Initial Q-values:

Episode 1 – Example

Suppose:

Action:

Reward:

Next state:

Assume:

Then:

Episode 1 – Second State-Action Pair

Suppose:

Reward:

and:

Then:

Episode 2 – Update

Suppose the agent again chooses:

with:

and:

Current value:

Then:

Thus, the Q-value increases because the action produced a positive reward.

Example Q-Table

State	Diet	Exercise	Medication	Monitor
S0	0.00	0.00	0.00	0.00
S1	0.30	0.77	0.20	0.10
S2	1.337	0.50	0.40	0.20
S3	0.20	0.30	0.90	0.10

These values are **illustrative** for demonstrating the Q-Learning calculation.

Training

The agent should be trained for at least:

During each episode:

1. Observe current patient state.
2. Select an action.
3. Receive the reward.
4. Observe the next state.
5. Update the Q-value.
6. Continue until the episode terminates.

Convergence Criterion

A suitable criterion is that the maximum change in the Q-table becomes sufficiently small:

For example:

for several consecutive episodes.

(c) Final Learned Policy

The policy is obtained by selecting the action with the highest Q-value:

Example:

Patient State	Best Action
S0 – Controlled	Monitor
S1 – Moderate Risk	Exercise
S2 – High Risk	Diet
S3 – Poorly Controlled	Medication*
S4 – Improved	Monitor

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

*Medication recommendations in an actual healthcare system must be controlled by qualified clinicians and applicable medical protocols.

Ethical Considerations

Deploying Reinforcement Learning for healthcare requires much stronger safeguards than a normal recommendation system.

1. Patient Safety

The system must not recommend unsafe treatments merely because they produce a high learned reward.

2. Human Oversight

A qualified healthcare professional should review treatment recommendations, particularly medication-related decisions.

3. Bias

If the training data is biased toward a particular demographic group, the agent may produce unfair or less accurate recommendations for other groups.

4. Explainability

The system should provide reasons for recommendations, such as:

High glucose + high BMI

↓

Elevated risk

↓

Recommended action: Lifestyle intervention

5. Data Privacy

Patient records contain sensitive information and must be securely stored and processed according to applicable healthcare privacy requirements.

6. Continuous Validation

The model should be monitored after deployment to detect performance degradation, unexpected behaviour, and population changes.